ЛЕКЦИЯ 2. ЭКОСИСТЕМА ANDROID

ANDROID SDK

- **SDK** (Software Development Kit) - набор инструментов для разработки
- Плагины для IDE (Android Studio)
- Инструменты сборки (aapt2, d8, r8)
- Инструменты отладки (adb, profiler)
- Библиотеки (AndroidX, Google Play Services)
- Образы эмуляторов (system images)

СТРУКТУРА ANDROID SDK

```
Android SDK/
├── platforms/
│   ├── android-34/
│   │   ├── android.jar
│   │   └── data/
│   └── android-35/
├── build-tools/
│   ├── 34.0.0/
│   │   ├── aapt2      # компиляция ресурсов
│   │   ├── d8         # конвертер в dex
│   │   ├── r8         # сокращение/оптимизация
│   │   └── zipalign   # выравнивание APK
│   └── 35.0.0/
├── platform-tools/
│   ├── adb            # Android Debug Bridge
│   ├── fastboot
│   └── sqlite3
├── emulator/
├── cmdline-tools/
```

```
└── ndk/                    # Native Development Kit
```

ADB (ANDROID DEBUG BRIDGE)

```
# Список подключённых устройств
adb devices

# Установка APK
adb install app.apk

# Удаление приложения
adb uninstall com.example.myapp

# Запуск Activity
adb shell am start -n com.example/.MainActivity

# Просмотр логов
adb logcat
adb logcat -s MyAppTag:D

# Копирование файлов
```

```
adb push local.file /sdcard/
adb pull /sdcard/remote.file ./

# Скриншот
adb shell screencap /sdcard/screen.png
```

GRADLE: СИСТЕМА СБОРКИ

- Построена на принципах Apache Ant и Apache Maven
- DSL использует **Kotlin** (build.gradle.kts) или Groovy
- Направленный ацикличный граф (DAG) для порядка выполнения задач
- Кэширование и инкрементальная сборка
- **Gradle Wrapper** (gradlew) - обеспечивает одинаковую версию

GRADLE: ТРИ ФАЗЫ ЖИЗНЕННОГО ЦИКЛА

1. INITIALIZATION

Определение проектов. Gradle читает `settings.gradle.kts` и определяет, какие модули (проекты) участвуют в сборке.

2. CONFIGURATION

Настройка всех задач. Выполняются все `build.gradle.kts` файлы. Создаётся граф задач.

3. EXECUTION

Запуск выбранных задач в правильном порядке (по DAG).

GRADLE: ЗАДАЧИ (TASKS)

```
// Объявление задачи
tasks.register("hello") {
    group = "custom"
    description = "Prints hello"
    doLast {
        println("Hello, Gradle!")
    }
}

// Запуск: ./gradlew hello

// Цепочка задач
tasks.register("deploy") {
    dependsOn("build", "test")
    doLast { println("Deploying...") }
}
```

GRADLE: КОНФИГУРАЦИЯ ЗАДАЧ

```
// Unsafe: выполняется при КАЖДОМ запуске
task slow {
    println("Это выполнится всегда!")
}

// Safe: выполняется ТОЛЬКО при вызове задачи
task fast {
    doFirst {
        println("Перед выполнением")
    }
    doLast {
        println("После выполнения")
    }
}

// Task type-ы
tasks.register<Copy>("backup") {
```

```
    from("src/main")
    into("backup")
}
tasks.register<Delete>("cleanBackup") {
    delete("backup")
}
```

GRADLE: ЗАВИСИМОСТИ

```
dependencies {
    // Реализация (попадает в APK)
    implementation("androidx.core:core-ktx:1.15.0")
    implementation("com.google.android.material:material:1

    // Только для компиляции (не попадает в APK)
    compileOnly("org.projectlombok:lombok:1.18.30")

    // Runtime только (не участвует в компиляции)
    runtimeOnly("org.slf4j:slf4j-simple:2.0.9")

    // Тестирование
    testImplementation("junit:junit:4.13.2")
    androidTestImplementation("androidx.test.ext:junit:1.2

    // Аннотации
    annotationProcessor("com.google.dagger:hilt-android-co
```

```
}
```

GRADLE: VERSION CATALOGS (2024+)

```
// gradle/libs.versions.toml
[versions]
kotlin = "2.0.21"
compose-bom = "2024.12.01"
room = "2.6.1"

[libraries]
core-ktx = { group = "androidx.core", name = "core-ktx", versi
compose-bom = { group = "androidx.compose", name = "compose-bo
room-runtime = { group = "androidx.room", name = "room-runtime

[plugins]
android-application = { id = "com.android.application", versio
kotlin-android = { id = "org.jetbrains.kotlin.android", versio

// build.gradle.kts
dependencies {
    implementation(libs.core.ktx)
    implementation(platform(libs.compose.bom))
```

```
    implementation(libs.room.runtime)
}
```

GRADLE: BUILD TYPES И FLAVORS

```
android {
    buildTypes {
        debug {
            isMinifyEnabled = false
            applicationIdSuffix = ".debug"
        }
        release {
            isMinifyEnabled = true
            isShrinkResources = true
            proguardFiles(
                getDefaultProguardFile("proguard-android-optim
                "proguard-rules.pro"
            )
        }
    }

    flavorDimensions += "environment"
    productFlavors {
        create("dev") {
```

```
            dimension = "environment"
            applicationIdSuffix = ".dev"
            versionNameSuffix = "-dev"
        }
        create("prod") {
            dimension = "environment"
        }
    }
}
```

GRADLE: ПРОИЗВОДИТЕЛЬНОСТЬ

- **Build Cache** - кэширование результатов задач
- **Configuration Cache** - кэширование конфигурации (Gradle 8+)
- **Incremental Compilation** - компиляция только изменённых файлов
- **Parallel Execution** - параллельная сборка модулей
- **Daemon** -фоновый процесс Gradle

```
# gradle.properties
org.gradle.daemon=true
org.gradle.parallel=true
org.gradle.caching=true
```

```
org.gradle.configuration-cache=true
org.gradle.jvmargs=-Xmx4g -XX:+UseParallelGC
```

ПРОЦЕСС ЗАГРУЗКИ ОС ANDROID

1. **Boot ROM** - загрузка из постоянной памяти
2. **BootLoader** - первичная инициализация
3. **Linux Kernel** - запуск ядра (Binder, ashmem, wakelocks)
4. **init()** - запуск init процесса
5. **Zygote** - процесс-предок всех Android-приложений
6. **system_server** - системные сервисы
7. **Activity Manager** - управление компонентами
8. **Launcher** - домашний экран

КЛЮЧЕВЫЕ КОМПОНЕНТЫ LINUX KERNEL

Компонент	Назначение
Binder	IPC (межпроцессное взаимодействие) - основа всех системных вызовов
ashmem	Анонимная разделяемая память - эффективный обмен данными
Low Memory Killer	Освобождение памяти при нехватке - убивает фоновые процессы
Wakelocks	Управление энергией/сном - предотвращает засыпание
Logger	Системное логирование (logcat)
ION	Управление памятью GPU - выделение буферов для отрисовки

PMEM Физическая разделяемая память

ZYGOTE: ПРОЦЕСС-ПРЕДОК

- Запускается при старте системы
- Инициализирует ART/Dalvik VM
- Загружает системные библиотеки
- **fork()** для создания нового процесса приложения
- Копирование виртуальной машины через copy-on-write
- Быстрый запуск приложений

```
# Zygote fork
Zygote (загружает VM, классы)
  ├── fork() → SystemServer
  ├── fork() → com.android.settings
```

```
├── fork() → com.example.myapp
└── fork() → com.google.android.gm
```

APK И AAB

APK (ANDROID PACKAGE)

- ZIP-архив с расширением .apk
- Содержит: AndroidManifest.xml, classes.dex, resources, lib/
- Установка напрямую (sideloading)
- Один APK для всех устройств

AAB (ANDROID APP BUNDLE)

- Формат для Google Play (с 2021 обязателен)
- Google Play генерирует оптимизированные APK
- Уменьшение размера на 15-50%
- Поддержка Dynamic Delivery

СТРУКТУРА APK

```
app-release.apk (ZIP-архив)
├── AndroidManifest.xml          # Манифест (бинарный XML)
├── classes.dex                  # DEX-код (Java/Kotlin → byte
├── classes2.dex                 # Второй DEX (при большом кол
├── resources.arsc               # Скомпилированные ресурсы
├── res/                         # Ресурсы (layout, drawable,
│   ├── layout/activity_main.xml
│   ├── drawable/icon.png
│   └── values/strings.xml
├── lib/                         # Нативные библиотеки (.so)
│   ├── armeabi-v7a/            # ARM 32-bit
│   ├── arm64-v8a/              # ARM 64-bit
│   └── x86_64/                 # x84 64-bit (эмулятор)
├── assets/                      # Необработанные файлы
├── META-INF/                    # Метаданные и подпись
│   ├── CERT.SF
│   ├── CERT.RSA
│   └── MANIFEST.MF
└── kotlin/                      # Kotlin-метаданные
```

```
└── kotlin-metadata-jvm.properties
```

ANDROID APP BUNDLE (AAB)

```
# Структура AAB
base.apk (AAB)
├── AndroidManifest.xml
├── BUNDLE-METADATA/
├── base/
│   ├── dex/
│   │   └── classes.dex
│   ├── res/
│   └── root/
│       └── resources.arsc
├── manifest/
│   └── AndroidManifest.xml
└── splits/
    ├── config_en.arm64_v8a.apk    # конфигурация
    ├── config_ru.arm64_v8a.apk
    ├── density_xxhdpi.apk          # плотность
    └── abi_arm64_v8a.apk           # архитектура

# Google Play Dynamic Delivery:
```

```
# Пользователь скачивает ТОЛЬКО нужные модули
```

ПРОЦЕСС СБОРКИ ANDROID-ПРИЛОЖЕНИЯ

1. **Компиляция Kotlin/Java** -> .class файлы (javac/kotlinc)
2. **D8** -> classes.dex (Dex-код для ART/Dalvik)
3. **R8** -> оптимизация, сокращение, обфускация
4. **aapt2** -> обработка ресурсов (resources.arsc)
5. **Манифест** -> AndroidManifest.xml (слияние с variant)
6. **Упаковка** -> APK/AAB
7. **Подпись** -> keystore (jarsigner / apksigner)
8. **Запуск** -> установка на устройство (adb install)

DALVIK VS ART

DALVIK (ANDROID 1-4)

- JIT-компиляция (Just-In-Time)
- Компиляция при каждом запуске
- Высокое энергопотребление
- Удалён в Android 5+

ART (ANDROID 5+)

- AOT-компиляция (Ahead-Of-Time)
- Компиляция при установке
- Профилирование + фоновая оптимизация
- Современный подход: гибридный JIT+AOT+PGO

JVM VS DALVIK VM

Особенность	JVM	Dalvik VM
Байт-код	.class файлы (Java bytecode)	.dex файлы (Dalvik bytecode)
Регистры	Стек-ориентированная	Регистр-ориентированная (экономичнее)
Оптимизация	Длядesktop/серверов	Для мобильных (энергоэффективность)
Классы	Один класс = один .class файл	Несколько классов в одном .dex
Конвертация	javac	d8/r8 (ранее dx)

ART: ГИБРИДНЫЙ ПОДХОД (2025)

1. **Фаза 1: JIT** - при первом запуске "горячие" методы компилируются JIT
2. **Фаза 2: Профилирование** - собираются данные о частых методах
3. **Фаза 3: Фоновая AOT** - системный процесс (bg-dexopt) компилирует в AOT

Результат: быстрый запуск + хорошая производительность + оптимальное энергопотребление.

```
# Профилирование ART
Включено: искусственно ускоренная компиляция (speed-profile)
Компиляция: методы из профиля -> AOST (остальные остаются JIT)

# Флаги ART
dalvik.vm.dex2oat-threads=4
```

```
dalvik.vm.image-dex2oat-threads=4
```

ART: GARBAGE COLLECTION

- Поколенческий сборщик мусора (Generational GC)
- **Concurrent copying collector** (Android 8+)
- Минимальные паузы (pasue < 1ms)
- Конкурентная работа (не блокирует main thread)

```
# Мониторинг GC
adb shell setprop log.tag.art GC D
adb logcat -s art:V

# Статистика памяти
```

D8 И R8: ИНСТРУМЕНТЫ СБОРКИ

D8

- Замена DX-компилятора
- Конвертирует .class -> .dex
- Поддержка Java 8+ language features
- Десугаринг (desugaring)
- Быстрее и компактнее DX

R8

- Замена ProGuard
- Сокращение кода (tree shaking)
- Оптимизация
- Обфускация
- Работает с .class и .dex

R8: СОКРАЩЕНИЕ КОДА (TREE SHAKING)

```
# Исходный код
class NetworkManager {
    public void connect() { ... }    // используется
    public void disconnect() { ... } // НЕ используется
    public void reconnect() { ... }  // НЕ используется
}

# После R8 (если connect() - единственная точка входа)
class NetworkManager {
    public void connect() { ... }    // оставлен
    # disconnect() и reconnect() УДАЛЕНЫ
}

# R8 проходит по графу вызовов от main()
# и удаляет недостижимый код
```

R8: ПРИМЕРЫ ПРАВИЛ (PROGUARD)

```
# proguard-rules.pro

# Сохранить конкретный класс
-keep public class com.example.NetworkManager {
    public *;
}

# Сохранить классы с аннотацией
-keep @com.google.gson.annotations.SerializedName class * { *; }

# Не сокращать enum
-keepclassmembers enum * {
    public static **[] values();
    public static ** valueOf(java.lang.String);
}

# Сохранить реализации Serializable
-keepclassmembers class * implements java.io.Serializable {
    static final long serialVersionUID;
    private void writeObject(java.io.ObjectOutputStream);
```

```
    private void readObject(java.io.ObjectInputStream);
}

# Обфускация имён
-repackageclasses ''
-allowaccessmodification
```

DESUGARING: JAVA 8+ API

```
# D8 позволяет использовать Java 8+ API на старых Android
# Например: java.time, java.util.stream, Optional

# build.gradle.kts
android {
    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_17
        targetCompatibility = JavaVersion.VERSION_17
        isCoreLibraryDesugaringEnabled = true
    }
}

dependencies {
    coreLibraryDesugaring("com.android.tools:desugar_jdk_l
}

// Теперь можно использовать java.time на Android 7+
```

```
val now = java.time.LocalDateTime.now()
val today = java.time.LocalDate.now()
```

KOTLIN В ANDROID-РАЗРАБОТКЕ

KOTLIN ANDROID EXTENSIONS VS VIEWBINDING

```
// Устаревший подход (Kotlin Android Extensions - удалён в
import kotlinx.android.synthetic.main.activity_main.*
myTextView.text = "Hello"

// Современный подход: ViewBinding
class MainActivity : AppCompatActivity() {
    private lateinit var binding: ActivityMainBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityMainBinding.inflate(layoutInflat
        setContentView(binding.root)
        binding.myTextView.text = "Hello"
    }
```

```
}
```

KOTLIN: VIEWMODEL В ANDROID

```
class MainViewModel : ViewModel() {
    private val _uiState = MutableStateFlow(UiState())
    val uiState: StateFlow = _uiState.asStateFlow()

    fun loadData() {
        viewModelScope.launch {
            _uiState.update { it.copy(isLoading = true) }
            val data = repository.fetchData()
            _uiState.update { it.copy(data = data, isLoadi
        }
    }
}

data class UiState(
    val isLoading: Boolean = false,
    val data: List = emptyList(),
    val error: String? = null
```

)

KOTLIN: COROUTINES В ANDROID

```
// Запуск корутины, привязанной к жизненному циклу Activity
lifecycleScope.launch {
    // Переключение на IO-поток
    val result = withContext(Dispatchers.IO) {
        apiService.getData()
    }
    // Возврат на Main-поток
    binding.textView.text = result
}

// Flow для реактивного наблюдения
class UserRepository {
    fun getUsers(): Flow> = flow {
        while (true) {
            val users = api.getUsers()
            emit(users)
            delay(5000)  // обновление каждые 5 секунд
        }
    }
```

```
}

// StateFlow для ViewModel
class UserViewModel : ViewModel() {
    val users: StateFlow> = repository.getUsers()
        .stateIn(viewModelScope, SharingStarted.WhileSubscribe
}
```

KOTLIN: DEPENDENCY INJECTION

```
// Hilt (Google) - DI для Android
@Module
@InstallIn(SingletonComponent::class)
object NetworkModule {
    @Provides
    @Singleton
    fun provideRetrofit(): Retrofit {
        return Retrofit.Builder()
            .baseUrl("https://api.example.com/")
            .addConverterFactory(GsonConverterFactory.create())
            .build()
    }
}

@HiltViewModel
class MainViewModel @Inject constructor(
    private val repository: UserRepository
) : ViewModel() { ... }
```

```
@AndroidEntryPoint
class MainActivity : AppCompatActivity() { ... }
```

ANDROIDMANIFEST.XML

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android

    <uses-permission android:name="android.permission.INTERNET" />
    <uses-permission android:name="android.permission.CAMERA" />

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/Theme.MyApp">

        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" /
                <category android:name="android.intent.category.LAU
            </intent-filter>
        </activity>
```

```
        <service android:name=".MyService" />
        <receiver android:name=".MyReceiver" />
        <provider android:name=".MyProvider" />

    </application>
</manifest>
```

RUNTIME PERMISSIONS (ANDROID 6+)

```
// Запрос разрешения (AndroidX Activity Result API)
val cameraPermission = registerForActivityResult(
    ActivityResultContracts.RequestPermission()
) { granted ->
    if (granted) {
        openCamera()
    } else {
        showPermissionDenied()
    }
}

// Проверка и запрос
if (ContextCompat.checkSelfPermission(this, Manifest.permissio
    == PackageManager.PERMISSION_GRANTED) {
    openCamera()
} else {
    cameraPermission.launch(Manifest.permission.CAMERA)
}
```

```
// Массовый запрос (Android 13+)
val multiplePermissions = registerForActivityResult(
    ActivityResultContracts.RequestMultiplePermissions()
) { permissions ->
    val allGranted = permissions.values.all { it }
}
```

GRADLE CONVENTION PLUGINS

- Способ общей логики сборки между модулями
- Создаются как **buildSrc** или отдельный модуль
- Подключаются через `plugins { id("my.convention") }`

```
// buildSrc/src/main/kotlin/android-library-convention.gradle.kts
plugins {
    id("com.android.library")
    id("org.jetbrains.kotlin.android")
}

android {
    compileSdk = 35
    defaultConfig {
        minSdk = 26
        testInstrumentationRunner = "androidx.test.runner.AndroidJU
    }
```

```
    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_17
        targetCompatibility = JavaVersion.VERSION_17
    }
}

// Модуль подключает:
plugins {
    id("android-library-convention")
}
```

GRADLE: KOTLIN DSL VS GROOVY

Особенность	Kotlin DSL (.kts)	Groovy (.gradle)
Типизация	Строгая типизация	Динамическая
Подсказки IDE	Полные (autocomplete, refactor)	Ограниченные
Производительность	Первый запуск медленнее (кэширование)	Быстрее на старте
Безопасность	Ошибки при компиляции	Ошибки только в runtime
Читаемость	Явный, предсказуемый	Краткий, но «магический»
Статус	Рекомендованный (2025+)	Устаревает

```
// Groovy
dependencies {
    implementation "androidx.core:core-ktx:1.15.0"
}
```

```
// Kotlin DSL
dependencies {
    implementation("androidx.core:core-ktx:1.15.0")
}
```

GRADLE PLUGIN PORTAL

- **plugins.gradle.org** - каталог плагинов Gradle
- Поиск плагинов по ID (например, `com.google.devtools.ksp`)
- Подключение в `settings.gradle.kts` или `build.gradle.kts`

```
// settings.gradle.kts
pluginManagement {
    repositories {
        gradlePluginPortal()
        google()
        mavenCentral()
    }
}

// build.gradle.kts
```

```
plugins {
    id("com.android.application") version "8.7.3" apply false
    id("org.jetbrains.kotlin.android") version "2.0.21" apply
    id("com.google.devtools.ksp") version "2.0.21-1.0.28" appl
}

// Или community-плагины:
plugins {
    id("com.github.ben-manes.versions") version "0.51.0"
    id("org.jlleitschuh.gradle.ktlint") version "12.1.1"
}
```

ANDROID APP LINKS

- Глубокая привязка (deep linking) к определённым экранам
- Безопаснее: проверяется владение доменом
- Открывается сразу в приложении (без диалога выбора)

```
<activity android:name=".ProductActivity"
    android:exported="true"
    android:launchMode="singleTask">
    <intent-filter android:autoVerify="true">
        <action android:name="android.intent.action.VIEW" />
        <category android:name="android.intent.category.DEFAULT" />
        <category android:name="android.intent.category.BROWSABLE" />
        <data
            android:scheme="https"
            android:host="example.com"
            android:pathPrefix="/product" />
```

```
    </intent-filter>
</activity>

// Обработка в Activity
val deepLinkIntent = intent
val productId = deepLinkIntent?.data?.getQueryParameter("id")
```

PROGUARD VS R8

Особенность	ProGuard	R8
Статус	Устарел	Активно развивается (Google)
Производительность	Медленнее	Быстрее в 2-3 раза
Размер APK	Больше	На 5-10% меньше
Интеграция	Отдельный шаг	Встроен в Android Gradle Plugin
Совместимость	.pro правила	Те же правила + расширения
Конфигурация	proguard-rules.pro	Тот же файл

```
// build.gradle.kts - R8 (по умолчанию в AGP 8+)
android {
    buildTypes {
        release {
            isMinifyEnabled = true
            isShrinkResources = true
            proguardFiles(
                getDefaultProguardFile("proguard-android-optim
```

```
                "proguard-rules.pro"
            )
        }
    }
}
```

R8 FULL MODE

- Агрессивная оптимизация для максимального сокращения
- Включается флагом `android.enableR8.fullMode=true`
- Удаляет неиспользуемые параметры методов
- Оптимизирует виртуальные вызовы
- Агрессивная обфускация имён классов

```
# gradle.properties
android.enableR8.fullMode=true

# Что делает R8 Full Mode:
# 1. Удаляет неиспользуемые аннотации
# 2. Конвертирует лямбды в статические методы
```

```
# 3. Оптимизирует switch-case в lookup-таблицы
# 4. Удаляет default-методы интерфейсов
# 5. Обфусцирует имена всех недостижимых классов

# Потенциальные проблемы:
# - Рефлексия может сломаться (нужны keep-правила)
# - Динамическая загрузка классов
# - Сериализация/десериализация
```

MULTI-MODULE ARCHITECTURE

- Разделение на модули: `app`, `:feature`, `:core`, `:data`
- Ускорение сборки (кэширование, инкрементальность)
- Изоляция зависимостей между модулями
- Повторное использование кода

```
// Структура проекта
:app                    # Application module
:feature:login          # Feature module
:feature:profile        # Feature module
:core:network           # Библиотека сети
:core:database          # Библиотека БД
:core:ui                # Общие UI-компоненты
:core:common            # Утилиты
```

```
// settings.gradle.kts
include(":app")
include(":feature:login")
include(":feature:profile")
include(":core:network")
include(":core:database")
include(":core:ui")
include(":core:common")
```

ANDROID BACKGROUND WORK

API	Use Case	Ограничения
WorkManager	Гарантированное выполнение (incluso перезагрузка)	Задержки до 15 мин, unreliable exact timing
JobScheduler	Планирование по условиям	API 21+, не guarantee выполнения
AlarmManager	Точные будильники	Android 12+: SCHEDULE_EXACT_ALARM permission
Foreground Service	Длительные задачи (>5 мин)	Требует уведомление, видимое пользователю

```
// WorkManager - рекомендуемый подход
class SyncWorker(ctx: Context, params: WorkerParameters) : Coroutin
    override suspend fun doWork(): Result {
        return try {
            syncData()
            Result.success()
```

```
        } catch (e: Exception) {
            Result.retry()
        }
    }
}

val syncRequest = OneTimeWorkRequestBuilder<SyncWorker>()
    .setConstraints(Constraints.Builder()
        .setRequiredNetworkType(NetworkType.CONNECTED)
        .setRequiresBatteryNotLow(true)
        .build())
    .setBackoffCriteria(BackoffPolicy.EXPONENTIAL, 30, TimeUnit.SEC
    .build()

WorkManager.getInstance(context).enqueue(syncRequest)
```

FOREGROUND SERVICES

- Выполняют задачи на переднем плане (album, maps, music)
- Требуют **уведомление** (notification) - обязательно
- Android 12+: нужно указать тип сервиса в манифесте
- Android 14+: ограничения для фоновых сервисов

```
// AndroidManifest.xml
<service
    android:name=".MusicService"
    android:foregroundServiceType="mediaPlayback"
    android:exported="false" />
```

```
// Запуск foreground service
val notification = NotificationCompat.Builder(this, CHANNEL_ID)
    .setContentTitle("Playing Music")
    .setContentText("Artist - Song")
    .setSmallIcon(R.drawable.ic_music)
    .build()

val service = Intent(this, MusicService::class.java)
ContextCompat.startForegroundService(this, service)

// В сервисе:
startForeground(NOTIFICATION_ID, notification)
```

NOTIFICATION CHANNELS

- Обязательны с Android 8.0+ (API 26)
- Пользователь может настраивать уведомления по каналам
- Один канал = один тип уведомлений

```
// Создание канала (вызывается при запуске)
val channel = NotificationChannel(
    "messages_id",
    "Messages",
    NotificationManager.IMPORTANCE_HIGH
).apply {
    description = "Incoming message notifications"
    enableVibration(true)
    vibrationPattern = longArrayOf(0, 100, 200)
}

val manager = getSystemService(NotificationManager::class.java)
```

```
manager.createNotificationChannel(channel)

// Отправка уведомления
val notification = NotificationCompat.Builder(context, "messages_id
    .setSmallIcon(R.drawable.ic_message)
    .setContentTitle("New message")
    .setContentText("Hello!")
    .setPriority(NotificationCompat.PRIORITY_HIGH)
    .setAutoCancel(true)
    .build()

NotificationManagerCompat.from(context).notify(1, notification)
```

BIOMETRIC AUTHENTICATION

- **BiometricPrompt API** - единый интерфейс для биометрии
- Поддержка: отпечаток, лицо, iris
- Биометрические данные хранятся в TEE (Trusted Execution Environment)
- Не хранит изображения отпечатков

```
val executor = ContextCompat.getMainExecutor(this)

val callback = object : BiometricPrompt.AuthenticationCallback() {
    override fun onAuthenticationSucceeded(result: BiometricPrompt.A
        super.onAuthenticationSucceeded(result)
        // Аутентификация успешна
        unlockApp()
    }
```

```
    override fun onAuthenticationError(errorCode: Int, errString: C
        super.onAuthenticationError(errorCode, errString)
        // Ошибка (например, отмена пользователем)
    }

    override fun onAuthenticationFailed() {
        super.onAuthenticationFailed()
        // Биометрия не распознана
    }
}

val promptInfo = BiometricPrompt.PromptInfo.Builder()
    .setTitle("Biometric Login")
    .setSubtitle("Verify your identity")
    .setAllowedAuthenticators(BiometricManager.Authenticators.BIOME
    .setNegativeButtonText("Cancel")
    .build()

val biometricPrompt = BiometricPrompt(this, executor, callback)
biometricPrompt.authenticate(promptInfo)
```

CAMERAX API

- Современная обёртка над Camera2 API
- Обеспечивает совместимость на 99%+ устройств
- Декларативный подход (Lifecycle binding)
- Встроенная поддержка: предпросмотр, захват, анализ

```
// build.gradle.kts
dependencies {
    implementation("androidx.camera:camera-core:1.4.1")
    implementation("androidx.camera:camera-camera2:1.4.1")
    implementation("androidx.camera:camera-lifecycle:1.4.1")
    implementation("androidx.camera:camera-view:1.4.1")
}

// Предпросмотр камеры
val previewView = findViewById<PreviewView>(R.id.previewView)
```

```
val cameraProviderFuture = ProcessCameraProvider.getInstance(this)
cameraProviderFuture.addListener({
    val cameraProvider = cameraProviderFuture.get()

    val preview = Preview.Builder()
        .setTargetAspectRatio(AspectRatio.RATIO_16_9)
        .build()
        .also { it.setSurfaceProvider(previewView.surfaceProvider) }

    val imageCapture = ImageCapture.Builder()
        .setCaptureMode(ImageCapture.CAPTURE_MODE_MINIMIZE_LATENCY)
        .build()

    cameraProvider.unbindAll()
    cameraProvider.bindToLifecycle(
        this, CameraSelector.DEFAULT_BACK_CAMERA, preview, imageCapture
    )
}, ContextCompat.getMainExecutor(this))
```

ANDROID STORAGE: SCOPED STORAGE

Версия	Хранилище	Доступ
Android 9 и ниже	READ/WRITE_EXTERNAL_STORAGE	Полный доступ ко всем файлам
Android 10-11	Scoped Storage (по умолчанию)	MediaStore + SAf
Android 12+	Scoped Storage (обязательно)	MediaStore + SAF + разрешения

```
// Чтение фото через MediaStore (Android 10+)
val projection = arrayOf(
    MediaStore.Images.Media._ID,
    MediaStore.Images.Media.DISPLAY_NAME,
    MediaStore.Images.Media.SIZE
)

val selection = "${MediaStore.Images.Media.SIZE} > ?"
val selectionArgs = arrayOf("0")

val cursor = contentResolver.query(
    MediaStore.Images.Media.EXTERNAL_CONTENT_URI,
```

```
    projection, selection, selectionArgs, null
)

// Storage Access Framework (SAF)
val intent = Intent(Intent.ACTION_OPEN_DOCUMENT).apply {
    addCategory(Intent.CATEGORY_OPENABLE)
    type = "image/*"
}
openDocumentLauncher.launch(intent)
```

ANDROID 14/15: НОВЫЕ ВОЗМОЖНОСТИ

ANDROID 14 (API 34)

- Foreground service types обязательны
- Ограничения на implicit broadcasts
- Полная поддержка Kotlin 2.0
- Улучшенная производительность (SlashGPU)
- Пермишн для точных будильников

ANDROID 15 (API 35)

- Private Space (приватное пространство)
- Двойной шифрование (credential-encrypted storage)
- Улучшения для планшетов/складных
- Edge-to-edge по умолчанию
- Improved garbage collection (ART)

```
// Android 14: Тип foreground service
<service
```

```
    android:name=".LocationService"
    android:foregroundServiceType="location" />

// Android 15: Edge-to-edge
WindowCompat.setDecorFitsSystemWindows(window, false)
```

ANDROID GAME DEVELOPMENT

- **GameActivity** (NDK) - нативная Activity для игр
- **GameTextInput** - ввод текста в играх
- **GamePerformance API** - мониторинг FPS
- Компоненты: SurfaceView, TextureView, GLSurfaceView

```
// build.gradle.kts
dependencies {
    implementation("androidx.games:games-activity:2.0.2")
    implementation("androidx.games:games-frame-pacing:2.0.1")
}

// Native Activity через C++
// app/src/main/cpp/native-lib.cpp
#include <game-activity/GameActivity.h>
```

```
void android_main(struct android_app* app) {
    while (true) {
        int events;
        SourceIdentifier* source;
        GameActivity_pollAllEvents(app, &events, &source);
        // Game loop: render, update, input
    }
}

// Производительность
val perf = getSystemService(GamePerformanceManager::class.java)
val fps = perf.currentFrameRate
```

MATERIAL YOU (MATERIAL 3)

- Динамическая палитра (Dynamic Color) на Android 12+
- Типографика, компоненты, цвета обновлены
- Темы на основе пользовательских обоев

```
// build.gradle.kts
dependencies {
    implementation("com.google.android.material:material:1.12.0")
    implementation(platform("androidx.compose:compose-bom:2024.12.0
    implementation("androidx.compose.material3:material3")
}

// Dynamic Color (Android 12+)
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
    val dynamicColor = dynamicLightColorScheme(this)
    MaterialTheme(colorScheme = dynamicColor) {
        // UI с цветами из обоев
```

```
    }
} else {
    MaterialTheme(colorScheme = LightColorScheme) {
        // Стандартная тема
    }
}

// Compose: Material 3 Composables
Button(onClick = { }) { Text("Button") }
Card { Content() }
NavigationRail { /* ... */ }
```

APP STARTUP LIBRARY

- Управление инициализацией компонентов при запуске
- Замена `ContentProvider` для init-кода
- Параллельная инициализация
- Зависимости между компонентами

```
// build.gradle.kts
dependencies {
    implementation("androidx.startup:startup-runtime:1.2.0")
}

// Кастомный Initializer
class AnalyticsInitializer : Initializer<Analytics> {
    override fun create(context: Context): Analytics {
        return Analytics.init(context)
    }
```

```
    override fun dependencies(): List<Class<out Initializer<*>>> {
        return listOf(FirebaseInitializer::class.java)
    }
}

// AndroidManifest.xml
<provider
    android:name="androidx.startup.InitializationProvider"
    android:authorities="${applicationId}.androidx-startup"
    android:exported="false"
    tools:node="merge">
    <meta-data
        android:name=".AnalyticsInitializer"
        android:value="androidx.startup" />
</provider>
```

BASELINE PROFILES

- Предварительная компиляция критических путей
- Ускорение первого запуска на 20-40%
- ART компилирует профиль при установке
- Генерируются автоматически (Macrobenchmark)

```
// build.gradle.kts
android {
    buildTypes {
        release {
            // ProGuard + Baseline Profile
            proguardFiles(...)
        }
    }
}

// Генерация профиля (тест-модуль)
```

```
@OptIn(ExperimentalBaselineProfilesApi::class)
class BaselineProfileGenerator {
    @get:Rule
    val baselineRule = BaselineRule()

    @Test
    fun generate() {
        baselineRule.collect(
            startup = true,
            maxIterations = 15,
            includeInStartupProfile = true
        ) {
            // Запуск приложения
            pressHome()
            startActivityAndWait()
            // Навигация
            device.findObject(By.text("Profile")).click()
            device.waitForIdle()
        }
    }
}

// Результат: baseline-prof.txt (precompiled profile)
```

ANDROID EMULATOR

- **Quick Boot** - быстрое восстановление (<6 сек)
- **Snapshots** - сохранение состояния
- **Google Play** - доступ к магазину
- **Extended Controls** -GPS, SMS, phone, battery

```
# Полезные команды
adb emu kill                          # Остановить эмулятор
adb shell settings list global        # Список настроек

# Управление эмулятором
adb shell am start -a android.intent.action.VIEW \
    -d "https://example.com"      # Открыть URL

# Установка сетевых ограничений
adb shell settings put global captive_portal_mode 0
```

```
# Быстрый запуск
emulator -avd Pixel_7_API_35 \
    -quick-boot \
    -no-snapshot-save \
    -gpu swiftshader_indirect \
    -memory 2048
```

ПУБЛИКАЦИЯ НА GOOGLE PLAY

1. **Подпись** - App Bundle (play-app-signing)
2. **Список** - скриншоты, описание, иконка
3. **Ценовая политика** - free / paid / in-app purchase
4. **Размер** - лимит 150 MB (AAB + OBB/Delivery)
5. **Review** - проверка 1-7 дней (автоматическая + ручная)
6. **Country/Region** - доступность по странам

```
# Build AAB для Google Play
./gradlew bundleRelease

# Signing
# Android Studio: Build > Generate Signed Bundle/APK
# Или через command line:
jarsigner -verbose -sigalg SHA256withRSA \
    -digestalg SHA-256 \
    -keystore release.keystore \
    app-release.aab release-key
```

```
# Загрузка на Play Console
# https://play.google.com/console
# Release > Production > Create new release > Upload AAB
```

СОВРЕМЕННЫЙ ANDROID-СТЕК (2025)

КОМПОНЕНТЫ

- **Activity** - экран
- **Fragment** - подэкран
- **ViewModel** - хранение состояния
- **Service** - фоновые задачи
- **WorkManager** - отложенные задачи

БИБЛИОТЕКИ

- **Room** - локальная БД
- **Retrofit** - сетевые запросы
- **Coil/Glide** - загрузка картинок
- **Hilt/Koin** - DI
- **Compose** - декларативный UI

ANDROID: ВЕРСИИ API (2025)

Версия	Кодовое имя	API	Доля устройств
Android 15	V	35	~5%
Android 14	Upside Down Cake	34	~20%
Android 13	Tiramisu	33	~25%
Android 12	S	31-32	~18%
Android 11	R	30	~12%
Android 10	Q	29	~10%

Рекомендация: minSdk = 26 (Android 8.0), targetSdk = 35